

GROUP ASSIGNMENT

TECHNOLOGY PARK MALAYSIA

CT115-3-3-DLI

DEEP LEARNING FOR INTRUSION DETECTION

HAND OUT DATE : 28TH FEBRUARY 2026

HAND IN DATE: 19TH APRIL 2026

WEIGHTAGE: 40%

GROUP MEMBERS:

Hemanthrajah A/L N Krishnarajah	**TP070844**
Anusha Devy A/P Brahmataran	**TP071154**
Gurshareenjit Kaur A/P Karamjit Singh	**TP070385**
Adam Jerome Joseph	**TP071194**
Sailess Raj A/L Rajendran	**TP070996**

Table of Contents

1.0 INTRODUCTION **3**

2.0 SECTION 1: INDIVIDUAL RESEARCH **4**

2.1 HEMANTHRAJAH A/L N KRISHNARAJAH 4

2.1.1 Enhancing ransomware defense: deep learning-based detection and family-wise classification of evolving threats (2024) *4*

2.1.2 Novel Ransomware Detection Exploiting Uncertainty and Calibration Quality Measures Using Deep Learning (2024) *6*

2.2 ANUSHA DEVY A/P BRAHMATARAN 8

2.2.1 Deep Learning-based Ransomware Detection Model with Hybrid Analysis *8*

2.2.2 Enhancing Ransomware Attack Detection Using Transfer Learning and Deep Learning Ensemble Models on Cloud-Encrypted Data *10*

2.3 GURSHAREENJIT KAUR A/P KARAMJIT SINGH 12

2.3.1 Early Ransomware Detection with Deep Learning Models (2024) *12*

2.3.2 RanSMAP: Open dataset of Ransomware Storage and Memory Access Patterns for creating deep learning-based ransomware detectors (2025) *14*

2.4 ADAM JEROME JOSEPH (TP071194) 15

2.4.1 Ransomware Detection Based on Machine Learning Using Memory Features (Research Paper 1) *15*

2.4.2 Zero-Day Ransomware Family Detection Based on Printable Character Analysis and Machine Learning (Research Paper 2) *16*

2.5 SAILESS RAJ A/L RAJENDRAN 18

2.5.1 XRan: Explainable Deep Learning-Based Ransomware Detection Using Dynamic Analysis. (2024) *18*

2.5.2 Ransomware Early Detection Techniques (2024) *19*

3.0 SECTION 2: ANALYSIS **20**

3.1 CRITICAL ANALYSIS 20

3.2 JUSTIFICATION FOR THE SELECTED PAPER 21

3.3 IMPLEMENTATION REPORT: CODE EXECUTION IN GOOGLE COLAB 22

3.4 GUI DEVELOPMENT AND FUNCTIONALITY 22

3.5 ANALYSIS OF MODEL PERFORMANCE BASED ON TESTS 28

3.6 OVERALL EVALUATION 29

4.0 CONCLUSION **30**

5.0 REFERENCES **31**

1.0 Introduction

Cybersecurity has gained significant importance in the field of computing due to the growing frequency, complexity and magnitude of modern attacks. Ransomware is one of these dangers as it can easily encrypt files, interfere with the working process, and lead to significant financial and operational losses within a relatively short time. The more familiar security tools like signature-based detection are still helpful, but tend to be less effective with newer or obfuscated ransomware variants. Due to this, researchers have been relying more on machine learning and deep learning methods to enhance detection of malicious behaviour using patterns and features and sequential data. The purpose of this assignment was thus to examine the efficacy of machine learning solutions to cybersecurity both as an individual research and as a group analysis and implementation effort.

Section 1 was a review and assessment of recent academic papers on machine learning techniques to address cybersecurity classification issues, by each group member. These papers were evaluated on the basis of their objectives, procedures, results, and practicality to the security goals. In Section 2, the group synthesized these findings to determine the existing trends in ransomware detection and to justify the choice of one apt paper to be implemented. The paper, Early Ransomware Detection with Deep Learning Models, was selected due to a good balance of its topicality, behavioural analysis, availability of open code, and the ability to be used in practice in Google Colab. According to this paper, the group was able to implement the source code, modify it to the Colab platform, and create the graphical user interface that enabled users to test the trained model with API sequences and uploaded CSV traces. In general, this report provides the theoretical and practical facets of machine learning application to ransomware detection, as well as demonstrates the benefits and shortcomings of these strategies in the real-world cybersecurity setting.

2.0 Section 1: Individual Research

2.1 Hemanthrajah A/L N Krishnarajah

2.1.1 Enhancing ransomware defense: deep learning-based detection and family-wise classification of evolving threats (2024)

Goals

The primary objective of the paper is to enhance ransomware detection through deep learning to identify malicious behavior in a better way. The more interesting aspect of this paper is that it is not only concerned with the detection of whether a sample is malicious or not, but also it attempts to categorize ransomware into various families. This can be important in the context of cybersecurity since knowing the ransomware family can assist the security analysts to better understand what type of threat they are dealing with and to respond more adequately. The primary goal of the paper is to discuss the issue of modern ransomware that is becoming more sophisticated and difficult to identify using the traditional signature-based mechanisms.

Methods

To this end, the authors came up with a Bidirectional Long Short-Term Memory model that is Group Normalization-based, also referred to as GN-BiLSTM. The model fits ransomware detection since ransomware behavior can be represented in a sequence, like the API calls, file access, and registry changes. The researchers analyzed the CIC-MalMem-2022 dataset and the self-collected dataset consisting of over twenty-one thousand samples comprised of malicious and benign files. They gathered behavioral characteristics with the help of Cuckoo Sandbox and could observe the behavior of the malware in execution instead of relying solely on the characteristics of the fixed files. The model proposed was also compared with other various deep learning models to find out whether the model was better than them.

Outcomes

The paper results were quite good and the GN-BiLSTM model is very high in detecting malware and also in the category and family-wise classification. This demonstrates that the model does not only work in detecting malicious behavior, but it also helps in discriminating between various ransomware families. I think it is one of the best arguments of the paper since it makes the research more applicable in a practical cybersecurity setting. But a point to keep in mind is that exceptionally high accuracy in research data does not imply that the model will behave similarly in practice in more complex and less controlled conditions of the real world. The other observable fact is that although the detection accuracy was extremely high, family classification was still more of a challenge, which implies that identifying specific ransomware families is a more complicated task.

Paper 1 Critical Analysis:

The strength of this paper lies in the fact that it is not merely the ransomware detection, but it tries to classify it family-wisely, which makes it more applicable in a real-life setting of a cybersecurity that requires an analyst to not just know that a threat exists, but what type of threat it is. The application of GN-BiLSTM is also suitable since ransomware behavior is dynamic and sequential in nature and therefore, a bidirectional model can be used to identify patterns compared to simple classifiers. The very high accuracy of the paper should be regarded with caution, though, as good results with controlled datasets do not necessarily translate to the same in real-world scenarios with increased variation, unseen attacks, and noise behavior. The next crucial limitation is that family attribution remained even harder than detection, which indicates that recognizing specific ransomware families is an even more complicated issue to tackle even with the overall detection being high.

2.1.2 Novel Ransomware Detection Exploiting Uncertainty and Calibration Quality Measures Using Deep Learning (2024)

Goals

This paper aims a bit differently compared to the first one since it is aimed not only at making the model more capable of detection of ransomware, but also at ensuring that the predictions made by it are more credible. One of the key points that the authors have raised about machine learning in cybersecurity is that, in some cases, a model may make a high-confidence prediction even when the prediction is incorrect. This can pose a threat to ransomware detection since a false alarm will enable a severe threat to pass unnoticed. Due to that reason, the paper will develop a model, which will be more effective in uncertainty management and will result in more reliable outcomes.

Methods

The model suggested by the researchers was known as Uncertainty-Aware Dynamic Early Stopping Deep Belief Network or UA-DES-DBN. Even though the name is very technical, the core concept is that the model makes use of information on uncertainty during training in such a way that it can learn in a more stable and reliable manner. The experiment integrates deep learning with the ideas of the Bayesian uncertainty, dropout-based estimation, and the calibration quality measures. Like the initial paper, API-call behaviour is the primary source of features used in this study that is suitable in ransomware detection since API calls may be used to understand how a suspicious program communicates with the operating system. The paper also examines the performance of the model in minimizing false alarms and increasing confidence in the predictions made, rather than just the overall accuracy of the model.

Outcomes

According to the paper, the suggested model was effective in detecting ransomware better than more traditional approaches. Not only did it have an improved overall accuracy, but it also reduced the false positive rate, which is noteworthy in the context of cybersecurity since

too many false alerts can consume the time of analysts and diminish their trust in the system. I personally believe that this paper is useful since it dwells on an issue that is ignored in most studies, and that is reliability of machine learning decisions. In practice, a model in cybersecurity must be not only accurate, but it must be reliable. A limitation, though, is that the model is built on Deep Belief Network, which is not as widely used in recent deep learning studies as other models, including LSTM or Transformers. It also appears to be more concerned with prediction quality as opposed to more practical deployment concerns like speed and efficiency.

Paper 2 Critical Analysis:

The strength of the paper is because the detection accuracy is not the only issue that has been addressed, but also reliability and trust in the predictions of the models which is highly crucial in the field of cybersecurity where false confidence is a risk. The ability to be in the gray and to be calibrated is an obvious advantage, as reducing the number of false positives and enhancing confidence in the predictions can make a detection system more useful to an analyst. Even so, the paper also has some limitations. The fact that it uses a Deep Belief Network predisposes it to be somewhat less consistent with the more popular modern ransomware research direction where LSTM, CNN, and Transformer-based models are more prevalent. Also, although the paper demonstrates great improvement of accuracy and false positive reduction, it does not seem as much concerned with such deployment factors like speed, scalability, and operational efficiency that are also relevant to the real-world implementation.

2.2 Anusha Devy A/P Brahmataran

2.2.1 Deep Learning-based Ransomware Detection Model with Hybrid Analysis

The primary objective of the study was to come up with a ransomware detection system to enhance better accuracy through the integration of both behavioural and static analysis. Most of the current systems use a single type of analysis, which does not help them to identify sophisticated ransomware. Thus, the research sought to fill this gap by uniting the two methods and comparing various architectures of Long Short-term Memory (LSTM) to identify the best model in detecting ransomware (Greish & Osman, 2025).

The framework proposed employed four models of LSTM such as Vanilla LSTM, Bidirectional LSTM (BiLSTM), Stacked LSTM, and Convolutional LSTM (ConvLSTM). Executable files were analysed to extract the static features (entropy and file structure) and behavioural features were gathered through system logs and API call sequences at runtime. A hybrid dataset was formed by combining these characteristics and trained using deep learning models as time-series data (Greish & Osman, 2025).

A fixed learning rate, batch size and optimisation algorithm have been used to train the models to provide a fair comparison. Performance metrics like accuracy, precision, recall and F1-score were used to evaluate it. This systematic method enabled the researchers to examine the performance of the various LSTM variants when used on hybrid ransomware detection tasks (Greish & Osman, 2025).

The findings indicated that ConvLSTM model performed better than all the other models, with an accuracy of 97.36% and high values of precision, recall, and F1-score. On the contrary, Vanilla, and Stacked LSTM models had lower recall, meaning that they failed to recognize high numbers of ransomware. The Bidirectional LSTM slightly improved but was yet again not as effective as ConvLSTM (Greish and Osman, 2025).

The report found out that hybrid analysis is much better in detecting ransomware than either using a static analysis or a behavioural analysis. Convolutional layers that were implemented in ConvLSTM allowed the model to adopt the spatial and temporal features. Nevertheless, the research also found some limitations including increased cost of computation

and dependence on the size of datasets. In spite of these difficulties, the suggested framework shows high capabilities of practical implementation in the sphere of cybersecurity systems (Greish & Osman, 2025).

Regarding a pragmatic approach to cybersecurity, the hybrid methodology employed in this study shows a great potential in enhancing ransomware detection, especially since it integrates both the static and behavioural analysis. This allows it to be more powerful in detecting advanced and adaptable ransomware that can avoid the traditional single-method systems. Nevertheless, the use of several LSTM-based architectures, as well as huge hybrid datasets, can provoke problems with scalability in real-world settings, particularly in scenarios where resources, such as computers, are scarce. Moreover, real-time deployment can be impacted because behavioural data collection and analysis entails processing. Nonetheless, the model remains very applicable to the contemporary cybersecurity applications due to these limitations, as it amplifies the detection accuracy and minimizes chances of undetected ransomware attacks.

2.2.2 Enhancing Ransomware Attack Detection Using Transfer Learning and Deep Learning Ensemble Models on Cloud-Encrypted Data

The main purpose of this study was to create a better ransomware detection model that could find encrypted and hidden malware in cloud environments. Static signatures and limited adaptability make it hard for traditional detection systems to keep up with modern ransomware. Consequently, this study sought to present a more precise and sophisticated methodology employing deep learning techniques capable of generalisation across various ransomware variants. The study also looked at how to make it easier to find encrypted data, which is becoming more common in cloud-based systems.

To accomplish this, the researchers suggested a hybrid deep learning framework called RANSOMNET+, which combines Convolutional Neural Networks (CNN) and Transformer architectures. The CNN part of the model extracted spatial features from the data, and the Transformer part extracted long-range dependencies and contextual relationships. Also, transfer learning was used to make the model work better and cut down on training time. The model learned complicated patterns linked to bad behaviour by using ransomware datasets that included both encrypted and harmless samples.

The study also used an ensemble learning strategy to bring together the best parts of several models. This method made detection more stable and reliable. In the training process, we utilized common performance metrics such as accuracy, precision, recall, and F1-score to identify features, enhance the model, and evaluate it. By using combining different learning methods the model captured both low-level and high-level features effectively.

RANSOMNET+ yielded a detection rate of almost 99.1% that was better than a range of (deep learning) models, such as ResNet50 and VGG16. The model had high precision and recall score, which indicated that the model was able to detect ransomware without a large number of false positives. It means that the joint CNN-Transformer architecture is highly beneficial in terms of ransomware detection.

Overall, the study concluded that the deep-learning methods and, specifically hybrid and ensemble models are ideally positioned to respond to the prevailing cybersecurity threats.

However, this model may be rather complicated and thus may require a lot of computing power. Despite this shortcoming, the proposed approach can be a viable initial step into any viable ransomware detection system, and it demonstrates the applicability of advanced deep learning techniques in the field of cybersecurity.

Viewing RANSOMNET+ through the prism of cybersecurity applications, it is a highly viable option of ransomware detection, particularly in cloud settings where encrypted data is prevalent. The CNN, Transformer, and ensemble learning are effective at the combination of detailed patterns and the global contextual relationships, which makes the model more resilient to more advanced and obfuscated attacks. Nevertheless, this model can be complicated and therefore it might not be applied in real-time systems because of the steep computational and resource demands. This may be a problem with large scale cloud systems where response time and efficiency are paramount. However, the model demonstrates great opportunities to improve the ransomware detection functionality, especially in the context of high-risk settings, where the accuracy of the processing is valued over the cost.

2.3 Gurshareenjit Kaur A/P Karamjit Singh

2.3.1 Early Ransomware Detection with Deep Learning Models (2024)

The first paper focuses on detecting ransomware during its early execution stage before encryption causes serious file damage. Davidian et al. (2024) note that conventional anti-malware tools are inefficient at detecting zero-day ransomware due to their heavy reliance on static signature approaches. To address this limitation, the authors proposed behavioural detection using API call sequences produced during software execution. API calls facilitate various operating system actions like file browsing, data storage, document editing, and system resource manipulation. It is common practice to detect malware based on its recurrent system interaction characteristics via an API (Sgandurra et al., 2016).

The authors used the technique of supervised learning to distinguish ransomware from a pool of ransomware consisting of 62 ransomware samples and 62 non-malicious samples. The researchers examined various AI models such as Deep Neural Networks (DNN), Convolutional Neural Network (CNN), and LSTM. Various context windows were tested to identify the optimal number of API calls that could be used as input features. The analysis revealed that seven API calls constituted the optimal context window size, and the CNN and LSTM models were superior to previous techniques since they accounted for sequential behavioral patterns better (Davidian et al., 2024).

It was noted that incorporating API operation-result information increased precision by enabling the model to learn the sequence of actions and their success. CNN and LSTM achieved high detection accuracy while maintaining prediction speed suitable for real-time deployment. This supports the growing role of deep learning in cybersecurity, as sequence-based neural models are effective in identifying complex attack behaviour (Vinayakumar et al., 2019; Davidian et al., 2024).

This paper is practical because API monitoring can be implemented using standard software tools. It is also easier to reproduce in educational environments such as Google Colab.

However, continuous API monitoring may increase computational overhead in large systems, and advanced ransomware may attempt to imitate benign software behaviour.

2.3.2 RanSMAP: Open dataset of Ransomware Storage and Memory Access Patterns for creating deep learning-based ransomware detectors (2025)

The second paper introduces RanSMAP, a ransomware detection dataset based on low-level storage and memory access patterns. Hirano and Kobayashi (2025) claim that software-level behaviour alone may not be sufficient because advanced ransomware can imitate normal application activity. Therefore, the researchers proposed collecting storage and memory activity below the operating system level using a lightweight hypervisor. A hypervisor-based approach is desirable since it records hardware-level activity that cannot be spoofed by adversaries (Hirano & Kobayashi, 2025).

The RanSMAP dataset has 11,820 CSV files created during 1,970 runs using ransomware samples, benign applications, and ransomware variants like Conti and REvil. Each execution produces six CSV files representing storage reads, storage writes, memory reads, memory writes, memory read-write activity, and execution patterns (Hirano & Kobayashi, 2025). These values are converted into 23-dimensional feature vectors containing five storage-related features and eighteen memory-related features. The deep learning architecture combines one-dimensional CNN with two LSTM layers for classification.

The results showed that combining storage and memory features improved ransomware detection compared with using storage features alone. The study reports a 2.3% improvement in F-score when both feature groups were applied together. Detection performance also improved when ransomware was executed simultaneously with standard applications such as Firefox and Office, which better reflects real user behaviour (Hirano & Kobayashi, 2025).

This paper offers stronger technical defence because hardware-level activities are harder for attackers to hide. It is especially useful against ransomware evasion strategies that imitate normal software behaviour. However, hypervisor-based deployment is more complex than software-level monitoring and may require specialised infrastructure.

2.4 Adam Jerome Joseph (TP071194)

2.4.1 Ransomware Detection Based on Machine Learning Using Memory Features (Research Paper 1)

The main goal of this study was to create a machine learning model that can use memory dump traits to find ransomware. The writers of this paper wanted to solve the issue where older ways of finding malware, like signature-based methods, do not work on newer ransomware that hides itself by encrypting and obfuscating data. They did not use known fingerprints, instead they looked at what happens in system memory when ransomware is run.

For the method, it took the experts about 7 months to build their own dataset from scratch. They got 579 samples of clean software from trustworthy sources and 586 samples of ransomware from sites like VirusShare and Bazar. The ransomware examples came from families like LockBit, BlackCat, and Conti. All the tests were being ran in a secure Cuckoo Sandbox that was running on Ubuntu and had a Windows 7 guest machine. Using a better version of the VolMemLyzer tool, they took out 58 memory features after running each sample for 300 seconds. Some of these features were process handles, loaded DLLs, API hooks, and code leaks. Then, they used Chi-Square and Sequential Forward Selection to pick out the most important features. After that, they tried five machine learning classifiers which were XGBoost, LightGBM, Random Forest, Extra Tree, and AdaBoost. Overall, XGBoost did the best, with a 97.85% success rate and a 2% false positive rate when 47 traits were used. But what was more interesting was that Random Forest was able to get 97% accuracy with only 16 features. This is a big performance boost because fewer features mean faster processing.

Security wise, this method has a lot of potential because memory analysis is harder for attackers to get around than static methods. Ransomware can easily change the identity of a file, but it acts more consistently when it is in memory. But memory dump analysis has a clear flaw, it needs malware to run first. This means it is not great for real-time protection. It is more of a tool for finding malware after it has occurred. Also, Windows 7 was used to make the dataset, which is a pretty old version of Windows now. It would be good to try the results on

more recent operating systems to see if they still hold up. Even though the sandbox is more secure, it might not fully reflect the real world, where viruses might act differently.

2.4.2 Zero-Day Ransomware Family Detection Based on Printable Character Analysis and Machine Learning (Research Paper 2)

The goal of this study was to use static analysis to find zero-day ransomware families, which means ransomware that has never been seen before. The goal was to create a simple way to find malware that does not require it to run, so it can be used in real time on devices with limited power.

The researchers used a dataset of 2,675 usable samples. The dataset was split into two sets. First, a training set with 2,157 samples from 25 known ransomware families and 1,134 samples of good software was used. Then, a test set with 518 samples from 15 different ransomware families and 133 samples of good software. As part of the Detection of New Ransomware Families (DNRF) method, the test families are never seen during training, so this split is very important. They used regular expressions to get readable letters from the binary files for feature extraction. They then made N-gram sequences and used TF-IDF to rank them. They tried nine ML models and used Variance Threshold to cut down on the number of features. Statistical tests like Friedman and Nemenyi were used to choose the end model, which was a soft-voting ensemble of Adaptive Boosting, Extra-Trees, and Logistic Regression. The group model got a 94.74% F1-score, 95.88% accuracy, 100% recall, and 90.50% precision. Making a guess needed only 0.45 seconds for each sample, and the model was only 1.3 MB.

The best thing about this method, in my opinion, is that it has a 100% recall rate, which means that it has never missed a real ransomware sample. This is very important for security, since even one missed threat can do a lot of damage. It works even if you do not run the file, which is another big positive for using it as a first line of security. But for some groups, like Maui and Quantum, the accuracy was pretty low, which tells us a lot of good files were marked as bad. In the real world, this would lead to alert fatigue, which is a common problem in security operations. Another worry is that heavy obfuscation can defeat static analysis. For example, if

ransomware writers encrypt their printable strings, this method might not work anymore. Still, this method seems like a good way to add an extra layer of security on top of other tools.

2.5 Sailess Raj A/L Rajendran

2.5.1 XRan: Explainable Deep Learning-Based Ransomware Detection Using Dynamic Analysis. (2024)

The first paper is focused on solving the urgent shortage of transparency of deep learning models to identify ransomware, an issue that may cause security analysts to hesitate to entirely rely on automated judgment. Signature-based systems that depend on known threats are effective to detect them, but they have difficulty detecting new, obfuscated, or zero-day ransomware samples. To overcome this weakness, the authors suggested an extensive method of detecting behaviour based on dynamic analysis. When running in a virtual sandbox (Cuckoo Sandbox in this case), several dynamic properties can be seen, providing a different viewpoint on the active behaviour of the software. The researchers used three essential elements to enrich their feature space; Application Program Interface (API) call sequences, dynamic link libraries (DLLs) and mutual exclusions (Mutexes).

The researchers adopted a two-layered Convolutional Neural Network (CNN) to train with these concatenated sequences and identify ransomware with high accuracy. The researchers added Explainable Artificial Intelligence (XAI) models to address the intrinsically black-box nature of these deep learning models. They used Interpretable Model-Agnostic Explanations (LIME) to explain local, per-prediction explanations (which individual APIs or DLLs triggered a prediction on a particular file) and SHapley Additive exPlanations (SHAP) to explain global explanations about how the model made its overall decisions. Their results indicated that this algorithm, known as XRan, was more effective as it reached a True Positive Rate (TPR) of up to 99.4% and was able to distinguish between ransomware and both normal software and overall malware.

2.5.2 Ransomware Early Detection Techniques (2024)

In the second paper, the authors concentrate on the critical period of detecting ransomware in its initial stages when it is running to ensure they can capture the malicious program when it is encrypting files. Alhashmi et al. (2024) point out that the ability to respond timely and immediately is also essential to ensure that more files are not encrypted irreversibly, which, as it is simply not sufficiently considered by modern antivirus software or other outdated models based on signatures, is not possible with the latter. In order to realize realistic, real-world implementation where milliseconds count, the authors suggested comparing traditional machine-learning systems with more resource-intensive deep-learning systems. They directly aimed at ensuring the highest computational efficiency and accuracy.

The authors tested six machine-learning methods on the whole UGRansome dataset, which describes samples with 14 distinct features, including network protocols, network flow bytes, and Bitcoin transaction information. The models that were tested were Logistic Regression, Decision Tree, Naive Bayes, Random Forest, AdaBoost and XGBoost. The researchers did not only test these models on standard predictive measures such as accuracy, precision and recall, but highly on their computational performance in terms of build time, training time, classification speed, their overall computational time and Kappa statistic. Their results conclusively revealed that Random Forest, Decision Tree, and XGBoost were quite promising algorithms to detect ransomware at an early stage. These models demonstrated outstanding accuracies of 99.37, 99.42 and 99.48. Moreover, they were very efficient in their classification speed and thus have been very effective in active, real time detection cases that are able to classify threats despite noise and changes in data.

3.0 Section 2: Analysis

3.1 Critical Analysis

As a result of the group's analysis of the papers under study, the trend towards increasing complexity of machine learning solutions used in ransomware detection became apparent. Modern ransomware frequently changes, utilizes various forms of obfuscation, and its behaviour differs significantly from earlier families of malware. In addition, a considerable number of recent scientific publications are devoted to the use of behaviour-based models, sequence-aware architecture and machine learning algorithms that are capable to detect attacks at their initial stages based on the analysis of API call patterns and temporal behaviour. Thus, while completing the assignment, the group should also analyse current trends in scientific studies, discuss the choice of particular techniques and algorithms and perform the evaluation of model's characteristics and performance.

On this basis, it can be stated that deep learning models preserving sequential information were considered to be more relevant than classical classification techniques to be used in ransomware early detection. Thus, recent research on hybrid ransomware detection proves that convolutional and recurrent neural networks, including LSTM-based models and hybrid approaches such as CNN-LSTMs, demonstrate good results in malware classification due to their capability to retain the sequential nature of malware behaviour and API calls. Additionally, another recent publication demonstrates how the usage of ConvLSTM enables achieving balanced results in terms of ransomware detection. Both of these facts confirm the effectiveness of the current trend in ransomware research, which consists in the combination of both temporal and spatial analysis in malware detection process.

However, at the same time, not all high-performance papers are equally convenient for further implementation and usage during the assignment completion. Despite their impressive results, some studies used complicated pipelines and architectures, private datasets or other methods, making them non-conformable to the requirements for implementation in the conditions of the assignment. For instance, the models using advanced hybrid pipelines such as GN-BiLSTM or LSTM-RNN demonstrated exceptionally high accuracy levels, but they also

included greater training costs and required more advanced model structures and data processing. In accordance with the task formulation, only the studies with publicly available code running in Google Colab and with GUI interface were relevant.

3.2 Justification for the Selected Paper

As can be seen from the comparison, the group chose Early Ransomware Detection with Deep Learning Models as the implementation paper. The rationale behind this decision was that it satisfied both criteria – scientific and practical. Firstly, the work examines early detection of malware based on API-call sequences for a couple of seconds of process activity. Such approach is highly related to preventing ransomware as it allows detecting malicious code in time, before it can cause any serious damage. Secondly, the article does not describe one configuration of experiment but several, including feature subset, text representation, context window size, and machine learning/deep learning techniques. This choice provided the team with additional information, as the paper explained reasons for effectiveness of certain configurations rather than suggesting just one solution.

This work also met the group's expectations regarding context window size being optimal at 7, best-performing machine learning/deep learning approaches such as CNN and LSTM, and an increase in precision by adding result to the API operation. Furthermore, according to the paper's authors, the presented model configurations can be used in practice because authors proved that the best results could be obtained in less than one second after data preparation for the test set on the PARSEC-500 dataset. Thus, the paper provided valuable information about choosing between models based on practical considerations.

3.3 Implementation Report: Code Execution in Google Colab

The first step in implementing the code for the paper was done through executing it on Google Colab as per the guidelines. This started with mounting Google Drive, unzipping the project ZIP file, and loading the files for the benign and ransomware datasets. Thereafter, we corrected many path issues, which were necessary after noticing that the original implementation was based on a different environment since some paths were incorrect. Further modifications were made in order for the code to load datasets stored within zip folders. Finally, the model ran well and the entire pipeline followed through loading, preprocessing, training, and testing. In the above process, the execution of the paper code was achieved.

In implementing the paper code in Colab, certain modifications were done as part of the effort to ensure stability. Among others, the changes included adjusting file paths, disabling auto-run parts that did not play any important role in the project execution, unpacking the compressed ransomware traces, and separating some artifacts including the trained model, vocabulary, and probabilities. All those changes ensured that the prototype developed from the paper was operational and easy to execute multiple times. However, those modifications did not change the research question posed in the paper.

3.4 GUI Development and Functionality

Once the model was successfully executed, the group created a graphical user interface within Google Colab using Gradio. Rather than a basic demonstration form, the interface was structured as a ransomware detection security console. It featured a dashboard, a prediction page, a CSV upload page, and a batch analysis page. The dashboard presented key performance metrics, including accuracy, precision, recall, F1-score, and a confusion matrix generated from the model's stored test predictions. The prediction page enabled users to manually input API sequences, while the upload page allowed CSV-based testing as long as the file included an operation column. The batch analysis page supported testing multiple sequences at once. This design satisfied the assignment requirement for a GUI that accepts user input and tests model classifications, while also going beyond a minimal solution to create a more polished and user-friendly interface.

A key enhancement was that the final GUI did not depend on artificial rule-based outputs. Instead, it incorporated the saved trained model along with the preprocessing artifacts, ensuring that predictions were produced by the actual classifier. This improved consistency with the paper's objectives and strengthened the report, since practical testing was now directly tied to the trained model rather than a simplified mock layer.

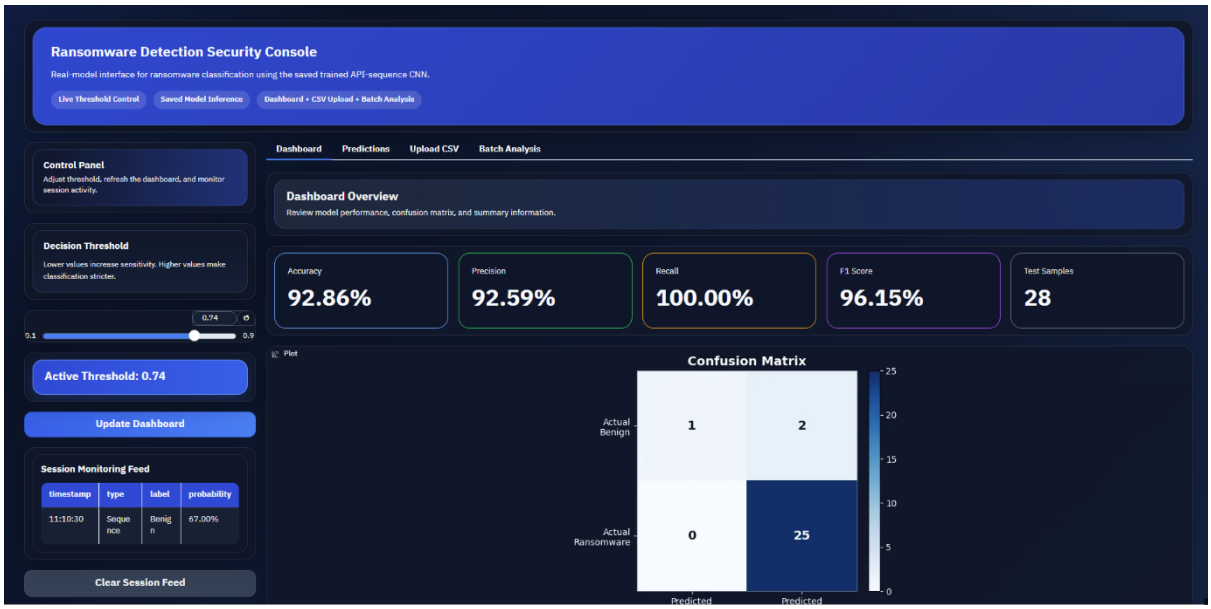


Figure 1: Dashboard



Figure 2: Dashboard Bottom page

GUI Interface Overview

The created GUI is a ransomware detector security console which enables the user to communicate with the trained model in a straightforward and orderly manner. The interface is separated into four major tabs, including Dashboard, Predictions, Upload CSV, and Batch Analysis. This architecture simplifies the system since each tab has a specific task, e.g. one can view model performance, test single API sequences, analyse uploaded trace files or test multiple sequences simultaneously.

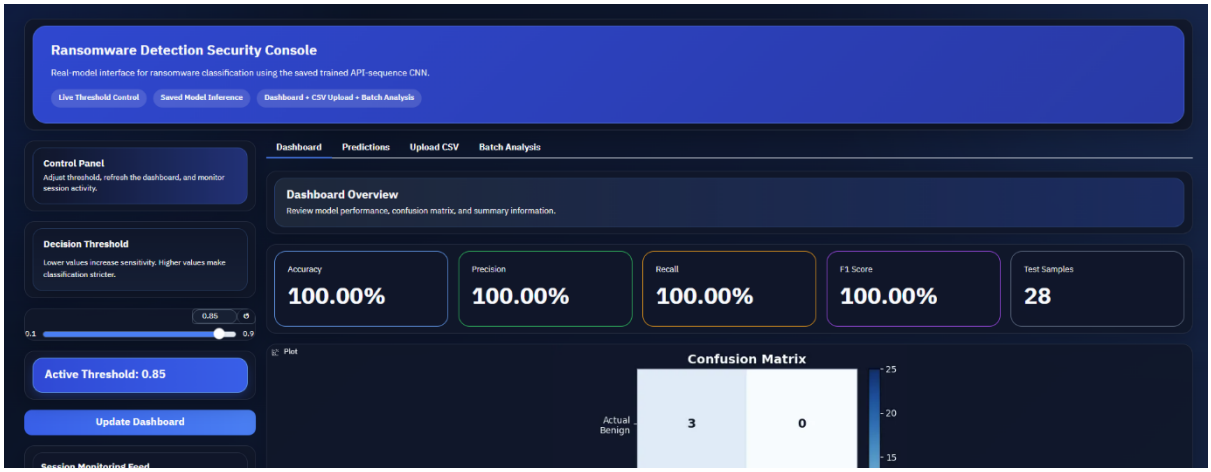


Figure 3: Dashboard with different threshold

Control Panel

The control panel on the left part of the interface provides the user with access to the global settings of the system. Here the Decision Threshold slider is the most significant. This is a threshold that is used to determine the strictness of the classifier in determining whether an input is benign or ransomware. When the threshold values are low, the model is more sensitive and when they are high, then the model is conservative. The current threshold is clearly displayed in the Active Threshold box and the Update Dashboard button updates the performance metrics and the confusion matrix to that specific threshold that has been selected. The control panel also has the Session Monitoring Feed that archives the last few prediction results that were done during the session and that assists the user to monitor previous tests without having to enter them again.

Dashboard Tab

The overall performance of the trained model is displayed in the Dashboard tab. It presents significant evaluation measures like Accuracy, Precision, Recall, F1 Score, and Test Samples. These values enable the user to get a quick idea of the performance of the model. Under the KPI cards, a Confusion Matrix is also displayed in the dashboard, and it visually describes the number of correctly and incorrectly classified benign and ransomware samples. The page is helpful as it provides the user with a brief overview of the model behaviour and eases the way of viewing the final performance outcome when the threshold is changed.



Figure 4: Prediction



Figure 5: Prediction Bottom Page

Predictions Tab

The Predictions tab enables the user to feed an API call sequence into the model in comma-separated format and run it on the trained model. Once an Analyze Sequence button is clicked, a classification result is generated by the system, like benign or ransomware. It also displays the Risk Score that provides a better understanding of the suspiciousness of the input. Below the interface shows Prediction Details, which comprises the predicted class, probability of ransomware, threshold used, sequence length, and the count of unique APIs. Recommended Actions section gives easy follow-up recommendations depending on the prediction outcome.

The frequency chart is also presented to visualise the frequency of the appearance of each API in the input sequence. This is what renders the prediction page helpful in causing the classification as well as comprehending the pattern of the input under examination.



Figure 6: Upload CSV



Figure 7: Upload CSV Bottom Page

Upload CSV Tab

The Upload CSV tab will enable the user to test a real trace file rather than typing the sequence. The user posts a CSV or CSV.GZ file which contains an Operation column, and the system reads the file with the trained saved model. After clicking the Analyze Uploaded File button, the interface will indicate the prediction of the uploaded trace as ransomware or benign. It also shows the ransomware probability, threshold taken and length of the sequence taken in the file. Like the Predictions page, this page will also contain recommended actions to ensure that the user is aware of how to interpret the result. This is useful since it brings the system

nearer to a realistic use case in the real world where logs or process traces can be directly analysed.

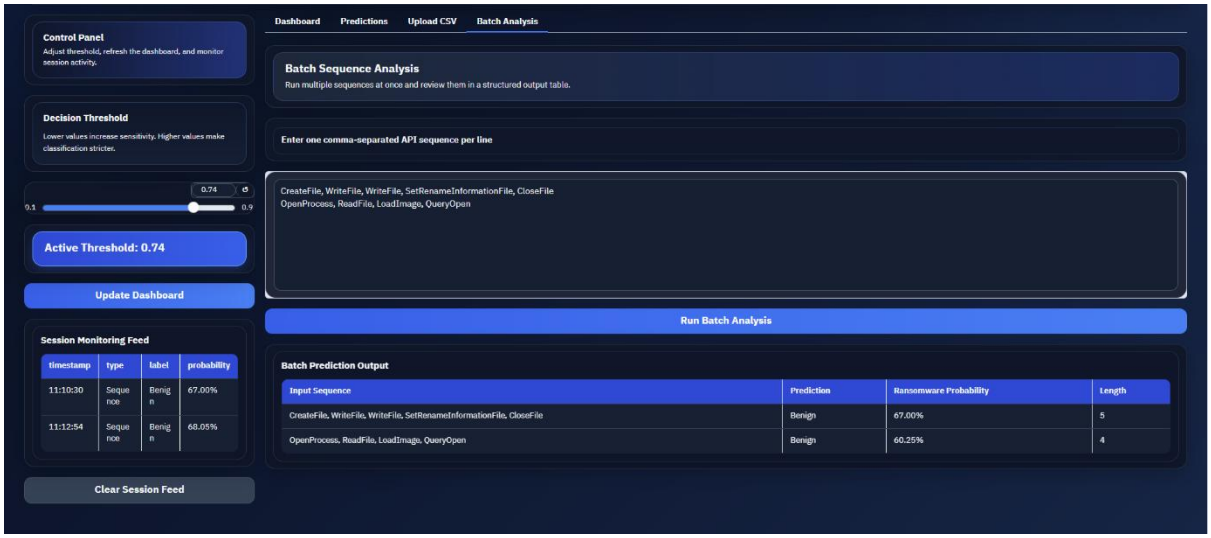


Figure 8: Batch Sequence Analysis

Batch Analysis Tab

The Batch Analysis tab is an API that can be used to test multi-api sequences. The user can input multiple line-separated comma-separated sequences instead of doing one at a time. Upon executing the batch analysis, the interface generates structured output table that contains the result of each input sequence, its predicted label, it ransomware probability and its length. The usefulness of this tab is that it enhances efficiency and enables the comparison of a number of cases beside each other, which is particularly helpful in case of experiments, demonstrations, or evaluation.

An overview of how the GUI works.

In general, the GUI operates by relating the input by the user to the one that is stored as a trained ransomware detection model. Whenever the user enters a sequence, or loads a CSV file or batch testing, the system will process the input into the format needed by the model first. The model will then generate a probability score which is compared to the selected threshold to generate the final classification. The result and supporting information such as the risk score, recommendation text and history of the session is then presented in the corresponding tab. This way, the GUI will not only display the ultimate prediction, but help the user make sense of the reasoning, and the actual meaning of the output.

3.5 Analysis of Model Performance Based on Tests

Nevertheless, the testing also revealed some important limitations of the implemented model that need to be acknowledged. To begin with, the dashboard results showed that the model achieved strong accuracy, recall, and F1-score values within certain threshold ranges. This supports the paper's main claim that early ransomware detection using behavioural sequences and deep learning is feasible. The confusion matrix and performance metrics were particularly useful in showing that the model was not simply memorising labels but was distinguishing between benign and malicious patterns in a meaningful way. In practical terms, this indicates that the trained model can detect ransomware-like behaviour from early API activity, without needing to wait for visible damage, which is a key goal of this research.

Nonetheless, the GUI tests also indicated that model performance is significantly influenced by the classification threshold selected. Adjusting the threshold in the dashboard modified the confusion matrix and performance metrics, indicating that the final results were influenced by the strictness or leniency of the decision boundary established. This is an important finding because it means the model does not have only one universally correct operating point.

If the threshold is low, that means the system becomes more sensitive which will help to decrease risk of false negative but increase the risk of generating a false alarm. A high threshold means a more cautious system, but it may lead to overlooking some suspicious sequences. It is important for cybersecurity because a too-permissive system will be unable to stop the ransomware attack promptly, while a too-restrictive system may generate too many warnings.

Another important takeaway from the testing was that some benign traces were still flagged as ransomware. For example, certain harmless inputs were predicted as malicious with fairly high confidence. This suggests that normal file activities like opening, editing, or renaming files can sometimes look similar to ransomware behaviour from the model's point of view. This isn't entirely surprising since the model focuses on patterns of behaviour rather than fixed signatures, but it does show that strong performance metrics don't always translate into perfect real-world results.

This discovery is important because, in actual cybersecurity settings, false positives are a serious concern. High recall is important since overlooking an actual ransomware process

can be very expensive, but if excessive benign instances are marked as harmful, users may lose trust in the system and analysts could spend unnecessary time looking into benign activities. Consequently, the model seems to be better suited as a smart early-warning system rather than as a conclusive automated authority. In general, the group's experiments validate the worth of the chosen paper, as the executed system evidently illustrates that early sequence-based detection can be achieved; however, they also indicate that actual implementation would necessitate additional adjustments, enhanced calibration, a broader range of benign data, and further verification against real-world workloads before the system could be relied upon in a completely operational setting. This renders the execution both effective and practical: it affirms the advantages of the document, while also illustrating areas where additional enhancement is required

3.6 Overall Evaluation

To conclude, the chosen paper was the best choice for Section 2 because of its current relevance, accessible code, a replicable experimental setup, and a distinct link between API-call actions and early detection of ransomware. The group's Colab execution and GUI creation met the practical requirements of the task, while the testing phase offered valuable insights into both the model's strengths and weaknesses. This aligns with the higher-band criteria for Section 2, where better scores are given for an effectively designed, user-friendly GUI and for analysis that connects practical outcomes to the claims presented in the paper.

4.0 Conclusion

To conclude, this assignment was a great experience to understand how machine learning and deep learning can be used to solve cybersecurity issues, particularly in the field of ransomware detection. On the personal research front, every group member reviewed recent scholarly literature and formed a greater insight on how the contemporary detection systems leverage behavioral features, sequence aware and deep learning structures to enhance classification results. The results of this research were evaluated more critically through the group analysis component to determine key research trends, assess how practical various methods were, and explain why only one of the implementation papers was selected in the end. In analysis of the group, it was observed that sequence based methods are becoming more and more relevant in detection of ransomware since sequence based methods are more apt at capturing the temporal and behavioral characteristics of malicious activity as compared to the simple signature based methods.

This understanding was further reinforced in the practical implementation phase when it demonstrated that it is possible to convert research ideas into a working prototype though not easily. The chosen model was implemented in Google Colab and adjusted to the dataset and environment and embedded into a working GUI that facilitated the dashboard viewing, prediction of the API-sequences, testing the CSV upload, and analysis in batches. Simultaneously, testing process also demonstrated such critical limitations as threshold sensitivity and the existence of false positives in case benign traces are similar to ransomware like behavior. This demonstrates that although the chosen model has potential as an early-warning system, additional tuning, validation, and improvement of the dataset would be necessary before it could be regarded as an entirely reliable tool in a real-world application. In general, the assignment revealed that machine learning is promising in the field of cybersecurity, yet its practical application requires not only proper accuracy of models, but also responsible implementation, understandability, usability, and critical analysis of practical outcomes.

5.0 References

1. Singh, A., Mushtaq, Z., Abosaq, H. A., Mursal, S. N. F., Irfan, M., & Nowakowski, G. (2023). Enhancing Ransomware Attack Detection Using Transfer Learning and Deep Learning Ensemble Models on Cloud-Encrypted Data. *Electronics*, *12*(18), 3899. https://doi.org/10.3390/electronics12183899
2. *View of Deep Learning-based Ransomware Detection Model with Hybrid Analysis*. (2026). Ijic.Utm.My. https://ijic.utm.my/index.php/ijic/article/view/587/349
3. Hussain, A., Saadia, A., Alhussein, M., Gul, A., & Aurangzeb, K. (2024). *Enhancing ransomware defense: deep learning-based detection and family-wise classification of evolving threats*. *PeerJ Computer Science, 10*, e2546. https://doi.org/10.7717/peerj-cs.2546

   Hussain, A., Saadia, A., Alhussein, M., Gul, A., & Aurangzeb, K. (2024). *Enhancing ransomware defense: deep learning-based detection and family-wise classification of evolving threats*. *PeerJ Computer Science, 10*, e2546. https://doi.org/10.7717/peerj-cs.2546
4. Gazzan, M., & Sheldon, F. T. (2024). *Novel ransomware detection exploiting uncertainty and calibration quality measures using deep learning*. *Information, 15*(5), 262. https://doi.org/10.3390/info15050262
5. Alhashmi, A. A. (June , 2024). *Ransomware Early Detection Techniques.* Retrieved from Engineering, Technology & Applied Science Research,: https://doi.org/10.48084/etasr.6915
6. Sibel Gulmez, A. G. (2024). *XRan: Explainable deep learning-based ransomware detection using dynamic analysis,*. Retrieved from https://doi.org/10.1016/j.cose.2024.103703.
7. Davidian, M., Kiperberg, M., & Vanetik, N. (2024). *Early ransomware detection with deep learning models*. Future Internet, 16(8), 291.
8. Hirano, M., & Kobayashi, R. (2025). *RanSMAP: Open dataset of ransomware storage and memory access patterns for creating deep learning-based ransomware detectors*. Computers & Security, 150, 104202.
9. Kharraz, A., Robertson, W., Balzarotti, D., Bilge, L., & Kirda, E. (2015). *Cutting the gordian knot: A look under the hood of ransomware attacks.*
10. Sgandurra, D., Muñoz-González, L., Mohsen, R., & Lupu, E. C. (2016). *Automated dynamic analysis of ransomware.*

11. Vinayakumar, R., Alazab, M., Soman, K. P., Poornachandran, P., & Venkatraman, S. (2019). *Robust intelligent malware detection using deep learning.*
12. Aljabri, M., Alhaidari, F., Albuainain, A., Alrashidi, S., Alansari, J., Alqahtani, W., & Alshaya, J. (2024). Ransomware detection based on machine learning using memory features. *Egyptian Informatics Journal*, *25*, 100445. https://doi.org/10.1016/j.eij.2024.100445
13. Pinto, K. K. G., Farias, E. S., Moreira, D. C., & Moreira, C. C. (2025). Zero-day ransomware family detection based on printable character analysis and machine learning. *Revista Eletrônica de Iniciação Científica em Computação*, *23*(1). https://doi.org/10.5753/reic.2025.6021